

Embedded Systems Emulation

- An Evaluation of Cycle Count Accuracy

Emulering av inbyggda system

William Hellstrand
Hugo Holmberg

Supervisor : Christoffer Holm
Examiner : Filip Strömbäck

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <http://www.ep.liu.se/>.

Abstract

Embedded systems development increasingly relies on software emulation to enable testing and prototyping without having access to the physical hardware. While modern emulators offer a practical advantage, how accurate they represent the physical hardware remains a questioned topic, particularly at the cycle level. This thesis evaluates the cycle count accuracy of two embedded hardware emulators, QEMU and Wokwi, and compares them to a physical reference system. The physical hardware consists of an ESP32-C3-Zero microcontroller connected to a MFRC522 RFID reader communicating via SPI. The different environments is running similar workloads consisting of an RFID read operation and mapping, this workload is run 1000 times for each environment. Neither emulator achieves reliable cycle count accuracy, with incomplete peripheral modeling identified as a primary contributing factor.

Acknowledgments

We want to give a special thanks to our supervisor Christoffer Holm who supplied us with the original idea for the thesis and the hardware needed to see it through, also for all the guidance and help along the way. We also want to thank our examiner Filip Strömbäck for all the insight on the scientific view of the thesis and for pointing us in the right direction by making us think a few extra steps and consider all aspects.

Contents

Abstract	iii
Acknowledgments	iv
Contents	v
List of Figures	vii
List of Tables	vii
1 Introduction	1
1.1 Aim	2
1.2 Research questions	2
2 Background	3
3 Theory	5
3.1 Embedded systems emulation	5
3.2 QEMU	6
3.3 Wokwi	6
3.4 PlatformIO	6
3.5 SPI	6
3.6 SPIFFS	7
3.7 RFID	7
3.8 Simulation accuracy and abstraction layers	8
3.9 Cycle count as a metric	8
4 Method	9
4.1 Research Approach	9
4.2 Experimental Setup	9
4.3 Validity	15
5 Results	17
5.1 Summary Statistics	17
5.2 Observations	19
5.3 Validity and Reliability	20
6 Discussion	21
6.1 Emulator Accuracy Relative to Physical Hardware	21
6.2 Accuracy Difference Between Emulators	22
6.3 Contribution of Incomplete SPI Peripheral Modeling	22
6.4 Secondary Findings	22
6.5 Implications and Limitations	23
6.6 Ethics and Sustainability	23
6.7 Method	24

7 Conclusion	26
Bibliography	28

List of Figures

2.1	The wiring diagram for the attendance tracker	4
4.1	Graphical representation of diagram.json	13
5.1	Mean cycle counts per environment across ID group sizes, all measurements included.	18
5.2	Mean cycle counts per environment across ID group sizes, outliers removed ($> 1.5 \times$ median).	19

List of Tables

5.1	Summary statistics for all measurements. Wokwi $\times 16$ applied to counteract hard-coded counter, see Section 5.2.3.	18
5.2	Summary statistics with outliers removed ($> 1.5 \times$ median). Wokwi $\times 16$ corrects for the 10 MHz counter register running at 160 MHz.	18



1 Introduction

Embedded systems are specialized computers designed to fill a certain function within a larger device. Today, they are found in about every area of technology there is, everything from microcontrollers in household appliances such as refrigerators and air conditioner units to medical equipment or real-time control systems in automobiles and aircrafts.

When developers are working on these systems, access to the specific hardware can be limited. This is due to cost, time or that it is logistically challenging due to the hardware not existing yet. A common and important technique to solve this is emulation, computer programs simulating software running on the presumed hardware. Emulators recreate the behavior of target hardware in software, enabling development and testing without the need to have the physical hardware. This helps to reduce cost and enables developers who may not have access to hardware still be able to work on it [1, 2].

Despite the advantages, emulation may not always be a perfect substitute for actual physical hardware. Emulators often target instructions on a host system with a different architecture and characteristics. This may cause metrics like cycle count to behave differently on emulated hardware. Due to the uncertainty of emulators, as mentioned by K. Pathak et al. [3] where they found that the mean error can be 19–23%, it is interesting to evaluate different emulators against each other and the physical hardware they are emulating. In this thesis, the two emulators QEMU and Wokwi are tested and evaluated. The target being emulated is a simple embedded system consisting of an RFID reader connected to a microcontroller.

1.1 Aim

The aim of this thesis is to test and evaluate the difference in accuracy of different hardware emulators. This report assesses both their variance to each other and to real physical hardware. The main objectives is the following:

- To investigate how accurately embedded hardware emulators represent physical hardware in terms of cycle count accuracy.
- To compare the cycle count accuracy between different embedded hardware emulators.

1.2 Research questions

The results of this thesis will be produced by answering the following research questions:

- RQ1 How accurately do the evaluated embedded hardware emulators represent a ESP32-C3-Zero microcontroller in terms of cycle count accuracy?
- RQ2 How does cycle count accuracy differ between the evaluated embedded hardware emulators?
- RQ3 How does incomplete peripheral modeling, specifically the SPI-based RFID interface, contribute to cycle count inaccuracy in the evaluated emulators?



2 Background

The hardware used for the testing in this thesis is built on the request from the Department of Computer and Information Science (IDA) at Linköping University (LiU). The purpose of this device is tracking attendance during non-mandatory teaching moments, i.e. lectures and lessons, to be able to keep statistics on attendance in comparison to grades. This will be done using a keycard in the form of their student card (the LiU-card) and by manually entering name/LiU-id the first time registering attendance.

For this, a simple device, consisting of an RFID-reader and an ESP32-C3 microcontroller, is built as seen in Figure 2.1. Running on this hardware is a program that maps data from a student access card to a number associated with that student and if the student does not have a number associated with it the program maps it to a new unique number. This system will provide a practical test platform for evaluation of accuracy in cycle count of different hardware emulators.

The ESP32-C3-Zero was chosen as the microcontroller for the target system based on multiple of its features, its availability and low cost. Another important reason is the fact that it uses the RISC-V instruction set, which support for emulation is actively being developed [4] and is a architecture which usage has increased drastically over the last couple of years [5, 6]. The final reason is the onboard Wifi and Bluetooth capabilities, which in the future could be used as a mean to extract the attendance registration from the device to save on another device, such as a computer or a phone.

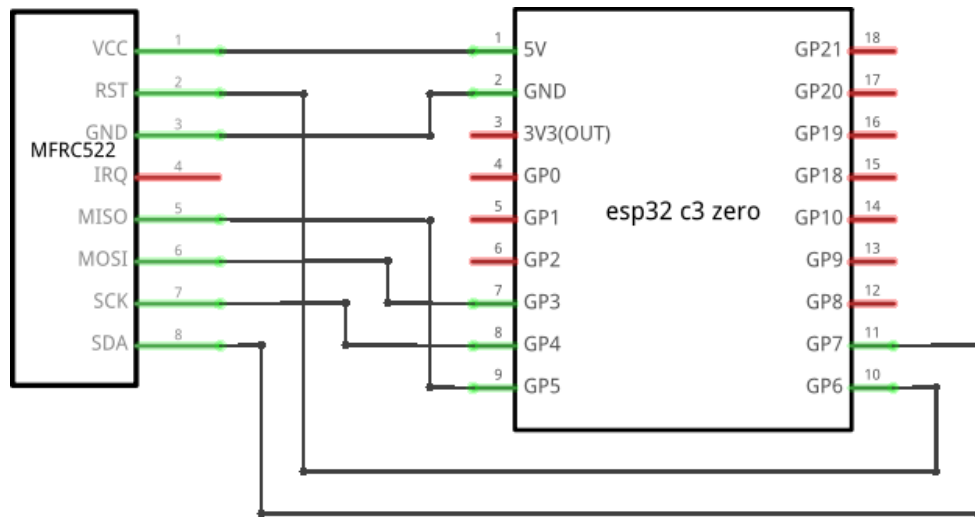


Figure 2.1: The wiring diagram for the attendance tracker



3 Theory

This section of the paper establishes a theoretical foundation for understanding embedded systems emulation and presents earlier research done in this or adjacent fields.

3.1 Embedded systems emulation

Emulation refers to the process of simulating hardware behavior in software. Emulators for embedded systems simulate the microcontroller or microprocessor, its memory and communication interfaces.

The accuracy of an emulation, meaning how closely it reproduces the timing and behavioral characteristics of the physical hardware, depends heavily on the fidelity of its hardware model. Recent research [3] shows that even carefully calibrated simulators can achieve mean errors of 19-23% when compared to the real hardware. Inaccurate emulation can lead to hard-to-catch bugs that may not even be noticed until later. Understanding the different trade-offs between different emulation approaches is an essential process for making informed decisions about development workflows.

Prior work has evaluated the accuracy of cycle-level emulators by simulating execution against measurements from physical hardware. For example, Sung Woo Chung et al. [7] studied cycle-accurate simulation on ARM1136-based systems to evaluate if detailed models could show deviating results from real hardware execution due to simplified pipeline and memory models. They found that while cycle-accurate and cycle-approximate emulation or simulation models reduce variation in accuracy they do not eliminate it.

Emulation approaches can be broadly categorized by their level of detailed in the modeled hardware. At the simplest level, the emulator only recreates the basic functionality such as register content, memory values and the result for each instruction. For more accurate emulation the models needs to be more detailed and simulate things like pipeline stages, cache and bus transactions, this will come with a higher computational cost [8].

3.2 QEMU

Quick Emulator (QEMU) is a widely used emulation platform that uses dynamic translation to convert target processor instructions to host processor instructions at runtime. It supports both full system emulation, running an entire operating system on a virtual machine, and Linux user-mode emulation, running individual Linux processes compiled for another CPU [9]. The latter one is the one this paper will focus on.

QEMU's dynamic binary translation works by translating blocks of its target instructions into instruction that the host can run and caching the result. This is a technique known as just-in-time (JIT) translation [9]. This is what makes QEMU fast compared to interpretive emulators, which instead translate each instruction one by one. Though, this could mean that QEMU's timing behavior could differ more from physical hardware since the number of host instructions executed per target instruction could vary.

Böhm et al. [10] found, while testing their JIT dynamic binary translation (DBT) simulator, a typical variance of 1.4% and as much as 5% in cycle count compared to a verified cycle-accurate interpretive simulation mode.

A notable limitation of the Espressif QEMU fork is its partial peripheral support. The SPI interface required by the RC522 RFID reader is not implemented in the current fork, which necessitates a mock signal substitution during testing and is an expected source of cycle count inaccuracy (see Section 4.2.2 and RQ3).

3.3 Wokwi

Wokwi [11] is an web-based simulation platform for development of embedded systems that can be used to simulate Arduino-, ESP32- and STM32-based systems along with a library of other parts and peripherals. It delivers an easy to use visual representation of the hardware along with the ability to simulate code running on said hardware. Wokwi can also be run as an extension in VS Code directly integrated into an already existing project.

Contrary to QEMU, Wokwi is built for embedded development and include functionality to simulate common peripheral such as displays, sensors and other modules. Among these peripherals is the exact RC522 RFID-reader we have chosen for our physical hardware. This allow for end-to-end testing of the full system with an easy to understand visual representation. However, the accuracy of the modeled peripherals in Wokwi is not clearly documented, so how closely the SPI communication with the RC522 model matches its physical counterpart remains to be seen.

3.4 PlatformIO

PlatformIO [12] is a cross-platform, cross-architecture development tool. It also exists as an extension for Visual Studio Code and provide an integrated environment for building, uploading and debugging firmware across a wide range of microcontroller platforms and frameworks.

3.5 SPI

Serial Peripheral Interface (SPI) [13] is a synchronous serial communication protocol commonly used in embedded systems for communication between microcontrollers and peripheral devices over a short distance. Developed by Motorola, it operates in a master/slave configuration over the four connections Master Out Slave In (MOSI), Master In Slave Out

(MISO), Serial Clock (SCK) and Chip Select (CS). The master and slave signal exchange data simultaneously on each clock cycle, in this thesis the ESP32-C3-Zero acts as the master and the RC522 as the slave. The number of clock cycles consumed by the SPI transaction depends on the configured clock speed and the number of bytes transferred, this makes the level of detail in the SPI modeling an interesting factor in the cycle count accuracy of an emulator that has SPI modeled.

3.6 SPIFFS

Serial Peripheral Interface Flash File System (SPIFFS) [14] is a lightweight file system designed for use with SPI-connected flash memory on microcontrollers such as the ESP8266 and ESP32. It enables embedded devices to store and access persistent data directly on their onboard flash memory.

SPIFFS manages flash memory as fixed-size logical blocks that are further subdivided into pages. All files are stored in a flat namespace, meaning no subdirectory structure is supported, files are instead identified by path-like names. When a file is written or modified, SPIFFS does not overwrite existing data, but marks the affected pages as obsolete and writes updated data to new pages, a strategy called copy-on-write. Obsolete pages are reclaimed periodically through garbage collection. To extend the operational lifespan of the flash memory, SPIFFS distributes write operations evenly across all available pages through a wear-levelling mechanism, since individual flash cells degrade after a finite number of erase cycles.

3.7 RFID

A Radio Frequency Identification (RFID) as explained by V. Chawla et al. [15] is a communication system between two components: a reader and a tag. Where the latter one can be a passive tag which can be attached to access cards. Passive tags have no way to generate its own power and instead get it from the electromagnetic field produced by the reader. The type of passive tag we are interested in in this paper is the near-field coupling. They communicate with each other by manipulating the electromagnetic field which has a range, for near-field, up to 1 m.

3.7.1 Frequency and Standards

Different RFID systems operate at different frequency bands, each suited for different use cases [15]. Low-frequency (LF) systems operate at less than 135 kHz, these are commonly used in animal tagging. High-frequency (HF) systems operate at 13.56 MHz and is used for, among other things, access control. Our chosen RC522 RFID-reader operates at 13.56 MHz and implements the ISO 14443-A standard, which is a relevant standard for HF systems, for the communication protocol between the reader and the MIFARE-compatible student cards used at Linköping University.

There is two more types of RFID systems that exists, ultra-high-frequency (UHF) and microwave, which operates at 860-960 MHz and 2.45 GHz respectively.

3.7.2 MFRC522

The MFRC522 is an integrated circuit RFID reader developed by NXP Semiconductors operating at 13.56 MHz and implements the ISO 14443-A standard¹. The RC522 communicates with a host microcontroller via SPI (Serial Peripheral Interface, explained in Section 3.5) through which the host configures the reader's registers and initiates the card transactions. A single

¹<https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf>

RFID read operation involves multiple sequential SPI register accesses, each of which contributes to the total cycle count of the operation. This multiple step transaction structure is what could make the RC522 hard to emulate accurately, any deviation in how the peripheral responds to each register access will impact the measured cycle count. This is observed in [3] where the authors found that peripheral modeling gaps were a primary source of error in RISC-V simulation.

3.8 Simulation accuracy and abstraction layers

The accuracy of an emulation closely depend on the abstraction layer in which the simulation is run. Chung et al. [7] found that even a cycle-accurate simulator of an ARM processor still would deviate from the physical hardware, this happened because the simulator used a simplified pipeline model that failed to capture all stall conditions that exists in the physical processor. Similarly, Pathak et al. [3] observed that component-level calibration of a RISC-V simulator could reduce but it didn't eliminate errors since models of peripheral and memory subsystems was approximate. These findings would suggest that with a more detailed and refined emulator model the accuracy of that model improves. Complete accuracy would require an extremely high level of detail in modeling every part of every hardware component, that would be too demanding to compute and eliminate the practicality of emulation [8].

In the configuration done for this thesis, QEMU operates at the functional level, meaning it produces the correct register and memory values but doesn't model pipeline timing, bus transactions or cache behavior. Wokwi's level of abstraction is unclear since documentation of its timing model isn't public. The uncertainty of Wokwi's abstraction level is also a motivation for evaluating its accuracy, since if the abstraction level was known we could predict an expected accuracy from existing literature.

3.9 Cycle count as a metric

Cycle count refers to the number of CPU clock cycles that has elapsed during a segment of the execution. On the ESP32-C3, clock cycles are counted using Espressif's custom performance counter registers that is documented in the ESP-C3 Technical Reference Manual². The event register `mpccer (0x7E0)` is configured to increment for every clock cycle, the value of the cycle counter is read from `mpccr (0x7E2)`. From a users perspective, the value of the cycle counter can be accessed with the Espressif IoT Development Framework (ESP-IDF) function `esp_cpu_get_cycle_count()` which reads `mpccr` directly as shown in the source code³.

Butko et al. [16] evaluated an emulator called GEM5⁴ compared to a physical ARM Cortex-A9 platform in terms of execution time and they found errors in the range of 1.39%-17.94% depending on the workload. While execution time and cycle count are different distinct metrics, they both show the same underlying issue with simplified hardware models introducing inaccuracies in timing.

²https://documentation.espressif.com/esp32-c3_technical_reference_manual_en.pdf

³https://github.com/espressif/esp-idf/blob/master/components/riscv/include/riscv/rv_utils.h

⁴<https://www.gem5.org/>



4 Method

This chapter describes the research methodology employed to answer the research questions stated in Section 1.2.

4.1 Research Approach

Runeson and Höst [17] describes four empirical research methodologies for software engineering: survey, experiment, case study and action research. Controlled experiments are characterized as studies that measure the effect of manipulating variables with a fixed design and primarily quantitative data. This thesis follows that characterization with the independent variable being the execution environment (physical hardware vs. emulators and emulators against each other), the dependent variable being cycle count accuracy and consists of a fixed design across environments.

An approach of controlled experiments is also consistent with related work: Pathak et al. [3] evaluates RISC-V simulators by comparing results from controlled benchmarks on both physical and emulated hardware.

The main limitations of this approach is that the conclusions are bound by the chosen hardware platform and test workloads. Runeson and Höst [17] notes that controlled experiments trade realism for control. Even so, that is an acceptable trade-off for the scope of this thesis.

4.2 Experimental Setup

The experimental setup consists of a physical hardware platform built for the purpose described in chapter 2 and two emulation environments, QEMU and Wokwi, each running as close an approximation of the same code as their respective capabilities allow¹. Each environment executes the same RFID read workload and cycle count are collected for comparison, see Section 4.2.4.

¹The full source code is available at github.com/absquatulates/LIU-Attendance-Tracker [18]

4.2.1 Physical Hardware Platform

The physical reference system consists of an Esp32-C3 and an RFID RC522 reader that communicates with each other via SPI. The code for the hardware is written in C using the ESP-IDF for ESP-C3 processors then compiled and uploaded using PlatformIO, this for ease of use.

The ESP32-C3 runs by default at 160 MHz. Each trial is run 1000 times to allow for statistical analysis of measurements variance see Section 4.2.5.

4.2.2 QEMU

QEMU is used as a software emulator for the ESP32-C3. Since QEMU does not natively support the ESP32-C3, a patched fork maintained by Espressif² was used, which adds an ESP32-C3 machine model for the RISC-V instruction set. The Espressif QEMU fork adds machine support for the ESP32-C3's RISC-V core and memory map. Since the peripheral support in this fork is partial the SPI interface for the RC522 RFID-reader is not fully modeled, which is expected to be a source of inaccuracy in the cycle count measurements.

4.2.2.1 PlatformIO and ESP-IDF Configuration

PlatformIO is selected as the primary development environment, it is an open-source embedded development platform which was configured to target the ESP32-C3 DevKitM-1 board. The ESP-IDF was selected as the underlying framework, as it provides direct access to hardware peripherals, FreeRTOS task management, and low-level system APIs required for SPI communication with the RFID-RC522. The project build environment was defined in the PlatformIO configuration file (platformio.ini) as seen in Listing 1.

```
[env:qemu]
platform = espressif32
board = esp32-c3-devkitm-1
framework = espidf
monitor_speed = 115200
```

Listing 1: Platformio.ini configuration for QEMU environment

4.2.2.2 Compilation

The program was compiled with the command `pio run -e qemu`³ which internally calls the ESP-IDF CMake build system and the RISC-V GCC cross-compiler toolchain. The build process produces three binary artifacts: the second-stage bootloader (bootloader.bin), the partition table (partitions.bin), and the application firmware (firmware.bin), all placed in the environment-specific output directory `.pio/build/qemu/`.

4.2.2.3 Flash Image Preparation

QEMU requires a single flat binary image representing the complete contents of the SPI flash memory. This image was produced by merging the three build artifacts at their respective flash offsets using the `esptool` utility⁴ as shown in Listing 2.

²<https://github.com/espressif/qemu>

³The system installed binary was incompatible with `pio` we therefore had to use a more direct path `~/platformio/penv/bin/pio run -e qemu`

⁴<https://github.com/espressif/esptool>

The resulting merged binary was padded to 4 MiB as the ESP32-C3 machine model in QEMU requires flash images of a size that is a power-of-two multiple of one mebibyte. Padding was performed using standard Unix utilities as seen in Listing 3, filling unwritten regions with the byte value 0xFF to accurately represent the erased state of flash memory:

```
esptool --chip esp32c3 merge-bin -o merged.bin \  
  --flash-mode dio --flash-size 4MB \  
  0x0      bootloader.bin \  
  0x8000   partitions.bin \  
  0x10000  firmware.bin
```

Listing 2: The bash command for a single flat binary image

```
dd if=/dev/zero bs=1M count=4 | tr '\000' '\377' > flash.bin  
dd if=merged.bin of=flash.bin bs=1 conv=notrunc
```

Listing 3: The bash command for binary padding and filling unwritten regions

4.2.2.4 QEMU Execution

The emulator used was Espressif’s maintained fork of QEMU (version 9.2.2). The program was launched using the command and flags as shown in Listing 4. The `-nographic` flag routes all serial output to the host terminal’s standard input/output streams. The `-machine esp32c3` flag selects the ESP32-C3 board model, and the flash image is attached via the `if=mtd` interface, which corresponds to the SPI flash bus used internally by the machine model. The emulator boots the ESP-IDF second-stage bootloader, loads the application from the factory partition at offset 0x10000, and begins executing `app_main()` under the FreeRTOS scheduler.

```
qemu-system-riscv32 \  
  -nographic \  
  -machine esp32c3 \  
  -drive file=flash.bin,if=mtd,format=raw
```

Listing 4: The launch command and flags for QEMU emulation

4.2.3 Wokwi

Wokwi is used as an additional emulation environment as an extension directly integrated in the PlatformIO-project. Since the exact board used in the physical hardware, the ESP32-C3-zero, did not exist as an option a standard ESP32-C3 board was chosen, this since it uses the same processor and the only difference is the amount of available pins.

There already exist a modeled version of the MFRC522⁵, which allows for easier emulation of the SPI communication between the RFID-reader and the microcontroller.

During preliminary testing it was discovered that Wokwi's implementation of the MFRC522 model does not appear to support anti-collision at CASCADE levels beyond level 1, meaning it isn't able to read MIFARE cards with a 7-byte UID. The physical system used the LiU student card, which has a 7-byte UID, and QEMU does support this through the mock function. Since Wokwi could not do this the tests for Wokwi was run with a simulated 4-byte card instead.

To configure the emulation a config file named diagram.json, as shown in Listing 5, is created and consists of the hardware used and how they are connected. This also results in a visual representation, as seen in Figure 4.1, of the hardware, which could be edited in the web browser or in vscode with a paid license. Then a wokwi.toml file is needed to specify where the project to be emulated is, Listing 6.

```
{
  "version": 1,
  "author": "Hugo & William",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-c3-devkitm-1", "id": "esp", "top": 0, "left":
      ↪ 0, "attrs": {} },
    { "type": "board-mfrc522", "id": "rfid", "top": 16.55, "left":
      ↪ 214.64, "attrs": {} }
  ],
  "connections": [
    [ "esp:GND", "rfid:GND", "black", [] ],
    [ "esp:3V3", "rfid:3.3V", "red", [] ],
    [ "esp:3", "rfid:MOSI", "blue", [] ],
    [ "esp:4", "rfid:SCK", "yellow", [] ],
    [ "esp:5", "rfid:MISO", "green", [] ],
    [ "esp:6", "rfid:RST", "orange", [] ],
    [ "esp:7", "rfid:SDA", "purple", [] ],
    [ "rfid:3.3V", "esp:3V3.2", "green", [ "v-86.4", "h-217.02" ] ],
    [ "rfid:GND", "esp:GND.6", "black", [ "v57.6", "h-129.34" ] ]
  ],
  "dependencies": {}
}
```

Listing 5: Wokwi configuration for ESP32-C3 and MFRC522 - diagram.json

⁵<https://docs.wokwi.com/parts/board-mfrc522>

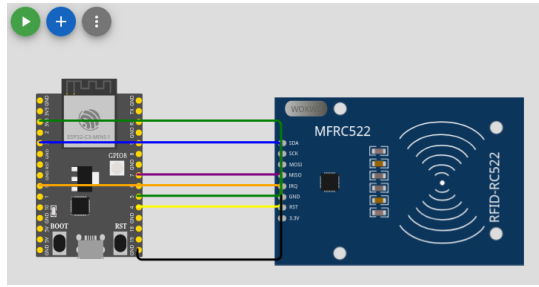


Figure 4.1: Graphical representation of diagram.json

```
[wokwi]
version = 1
firmware = ".pio/build/esp32-c3-devkitm-1/firmware.elf"
```

Listing 6: Wokwi project configuration - wokwi.toml

4.2.4 Testing

For testing, each version of the code will use the built-in ESP-IDF function `esp_cpu_get_cycle_count()` as shown Listing 7. This is triggered when the RFID-reader gets a reading. As mentioned in the chapter 3, on the ESP32-C3 clock cycles are counted using Espressif's custom performance counter registers. The event register `mpcer` (`0x7E0`) is configured to count clock cycles, and the count is read from the Machine Performance Counter Count register `mpccr` (`0x7E2`), which `esp_cpu_get_cycle_count()` reads directly. The elapsed count is calculated as an unsigned 32-bit subtraction, this so that any potential overflow can be handled. The same RFID tag is read for each physical trial, for the emulated tests an identical simulated tag response is triggered for each trial. The collected cycle counts is then used to calculate the mean, standard deviation, and relative deviation of each emulated environment from each other and from the physical hardware results. The cycle count measurements are logged in the terminal using `ESP_LOGI` for easy extraction.

```
#include "esp_cpu.h"

uint32_t start = esp_cpu_get_cycle_count();
//The artefact to measure
uint32_t elapsed = esp_cpu_get_cycle_count() - start;
```

Listing 7: Fetch cycle count

The firmware run during the testing performs the following sequence for each trial: it initializes the SPI bus and configures the MFR522 reader afterwards it starts to poll the reader until a card is detected. Upon detection it initializes the `start` variable, reads the card's UID over SPI, logs the UID using `spiffs`, then it reads the elapsed cycle count according to Listing 7. This sequence is repeated for 1 000 independent trials for each environment. For QEMU, where no SPI is modeled, the card UID is injected via a mock function that replicates the expected byte sequence.

The simulated workload were tested at three sizes 1, 5 and 10 IDs, to evaluate whether emulator accuracy remains consistent as workload increases. The different sizes were configured at the start to have that specified number of IDs logged. The 1-ID read represents the minimal workload as the program only have to check the scanned ID to the one stored and also only needing to log one ID. This makes it so that the 1-ID serves as a baseline for the minimal amount of stress the system could be put under. The 5-ID and 10-ID workloads were chosen as a more demanding workload for the reason that the program have to check more IDs to the scanned one and also that more IDs have to be logged, to reveal whether any deviation from physical hardware is roughly constant across workloads.

4.2.5 Analysis

Each environment, the workload is run for 1000 independent trials using the same RFID tag. Then the following statistics, explained in the next sections, are calculated:

- **Mean cycle count** (μ): the arithmetic mean of all 1000 trials, calculated according to equation (4.1).
- **Standard deviation** (σ): the spread of measurements within a single environment, calculated according to equation (4.2).
- **Emulation deviation from the physical hardware** (Δ): computed according to equation (4.3), expressing the deviation as a percentage.

Let c_i denote the cycle count measured in trial i , and n the total number of trials.

$$\mu = \frac{1}{n} \sum_{i=1}^n c_i \quad (4.1)$$

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (c_i - \mu)^2} \quad (4.2)$$

$$\Delta = \frac{\mu_{\text{emulator}} - \mu_{\text{physical}}}{\mu_{\text{physical}}} \quad (4.3)$$

Given the deterministic nature of the tested workload, a fixed RFID read operation on fixed firmware, large differences in standard deviation would be informative and interesting to discuss. All raw measurements are reported along side with the calculated statistics so that the results easily could be reproduced in the future.

4.2.6 Outliers

Since in preliminary testing, some trials resulted in a cycle count significantly higher than the norm the results will also be presented without this outliers to paint an additional picture of what happens on the system. These so called outliers are calculated according to equation (4.4).

$$c_i > 1.5 \times \text{median} \quad (4.4)$$

4.3 Validity

Following the earlier mentioned framework from Runeson and Höst [17], validity of the result is divided into four different classifications: construct validity, internal validity, external validity and reliability. The following subsections will discuss what threat exists for each classification of validity and what is done to counteract them, in the context of this thesis.

4.3.1 Construct validity

Construct validity concerns whether the the chosen metric actually measures something that will help answer the research questions. Cycle count is measured via the ESP-IDF function `esp_cpu_get_cycle_count()`, this metric captures timing behavior of the different environments at the instruction level which is relevant for accuracy in embedded systems emulation. However, only cycle count may not paint the whole picture of accuracy, metrics like execution time, interrupt latency or peripheral timing could help to compare the level of accuracy in more detail. With that said, cycle count lays a foundation for showing the accuracy on a level that is acceptable for the scope of this thesis.

4.3.2 Internal validity

Internal validity concerns whether the observed difference between the environments is caused by the given environment and not some other factor. The primary threat is the measurement itself, using the function to fetch the value of the cycle count register will by it self consumes a small number of cycles. But since the same function will be used in the same way in every version of the code the variation from environment to environment is likely negligible. Another threat could be non-determinism within the environment. During preliminary testing we found that QEMU could produce different results for the same workload. This is countered by running 1000 trials for each environment.

Another threat to the internal validity is the length difference in the UUIDs used in Wokwi. As described in Section 4.2.3, Wokwi was tested with a 4-byte UUID while the other two environments was tested using a 7-byte UUID. Since a 7-byte UUID requires an additional CASCADE level and therefore more SPI transactions the measured workload in Wokwi is not strictly identical to the other environments. This may contribute to an observed difference in the results.

4.3.3 External validity

External validity concerns whether the findings can be generalized beyond the specific conditions of this study. Since scope of this thesis is very narrow, only testing one metric within three environments, it's hard to argue that the results can be generalized. The testing is limited to one microcontroller, ESP32-C3-Zero, using the RISC-V instruction set and running a specific workload sending RFID data with SPI. The methodology itself can be ported to testing other workloads on other hardware platforms to further generalize the results, other metrics could also be added for broader findings.

4.3.4 Reliability

Reliability concerns whether the results could be reproduced and yield consistent results if the experiment were to be repeated. To strengthen the reliability of the results 1000 trials are run for each environment rather than a single measurement. By collecting a large number of data points the calculated mean becomes more stable and less sensitive to individual deviations, making the results more likely to be reproducible. A remaining threat to reliability is

potential non-determinism within the emulated environments, which cannot be fully eliminated. Running a higher number of trials would further improve reliability, but 1000 trials was determined to be sufficient for the scope of this thesis.



5 Results

This chapter presents the cycle count measurements collected from the tests of the three execution environments: QEMU, Wokwi and the physical hardware. Each environments was tested for 1000 trials using the same RFID read workload.

5.1 Summary Statistics

Table 5.1 presents the results from the testing by stating the mean cycle count (μ), standard deviation (σ) and the emulation deviation from the physical reference system (Δ) as defined in equation 4.3 for each environment. Figure 5.1 plots the mean cycle counts per environment across ID group sizes with all measurements included, illustrating how the environments diverge as workload grows.

As explained in Section 4.2.6 some of the results shows significantly higher cycle counts than the median, table 5.2 shows how many of these outliers showed up during the testing and what the mean, standard deviation and deviation from hardware it results in leaving these outliers out. Figure 5.2 plots the corresponding mean cycle counts with outliers removed, showing a cleaner picture of typical behavior across workload sizes.

Group	Environment	n	Mean (cycles)	Std dev (cycles)	Δ
1 ID	Physical HW	1 000	2 535 077	2 111 990	—
	QEMU	1 000	1 887 032	999 696	−25.6%
	Wokwi (raw)	999	109 406	32 279	−95.7%
	Wokwi ($\times 16$)	999	1 750 498	516 464	−30.9%
5 IDs	Physical HW	1 000	2 690 298	2 177 209	—
	QEMU	1 000	3 304 450	571 065	+22.8%
	Wokwi (raw)	999	110 256	32 279	−95.9%
	Wokwi ($\times 16$)	999	1 764 098	516 464	−34.4%
10 IDs	Physical HW	1 000	4 189 661	3 358 460	—
	QEMU	1 000	5 111 268	1 598 237	+22.0%
	Wokwi (raw)	999	132 430	53 462	−96.8%
	Wokwi ($\times 16$)	999	2 118 880	855 392	−49.4%

Table 5.1: Summary statistics for all measurements. Wokwi $\times 16$ applied to counteract hard-coded counter, see Section 5.2.3.

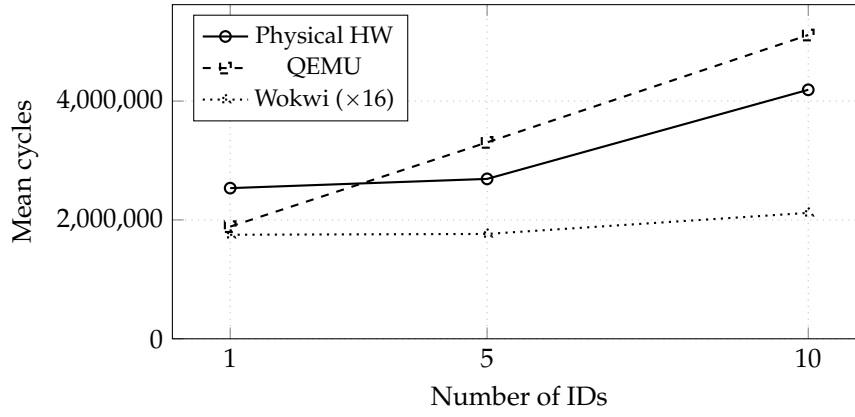


Figure 5.1: Mean cycle counts per environment across ID group sizes, all measurements included.

Group	Environment	n (clean)	Outliers	Mean (cycles)	Std dev (cycles)	Δ
1 ID	Physical HW	923	77	1 926 346	4 247	—
	QEMU	906	94	1 566 437	63 937	−18.7%
	Wokwi (raw)	922	77	100 078	35	−94.8%
	Wokwi ($\times 16$)	922	77	1 601 248	560	−16.9%
5 IDs	Physical HW	923	77	2 063 152	7 962	—
	QEMU	902	98	3 124 391	154 953	+51.4%
	Wokwi (raw)	922	77	100 928	35	−95.1%
	Wokwi ($\times 16$)	922	77	1 614 848	560	−21.7%
10 IDs	Physical HW	790	210	2 463 945	9 484	—
	QEMU	683	317	4 031 618	115 020	+63.8%
	Wokwi (raw)	789	210	104 912	38	−95.7%
	Wokwi ($\times 16$)	789	210	1 678 592	608	−31.9%

Table 5.2: Summary statistics with outliers removed ($>1.5 \times$ median). Wokwi $\times 16$ corrects for the 10 MHz counter register running at 160 MHz.

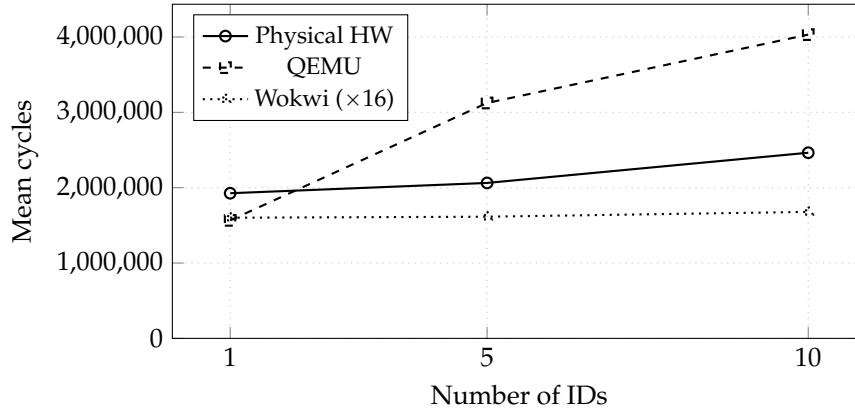


Figure 5.2: Mean cycle counts per environment across ID group sizes, outliers removed ($> 1.5 \times$ median).

5.2 Observations

Several patterns emerge from the summary presented in Table 5.2 and Table 5.1.

5.2.1 Physical hardware baseline

The physical hardware produces the most consistent measurements when the outliers are excluded with a standard deviation ranging from around 4 200 to 9 400 cycles. Contrary, when all the 1 000 measurements are included the standard deviation rises to between 2 100 000 and 3 400 000 cycles, showing the weight of the spikes from the outliers. The mean cycle count scales with the workload in both tables. Using all the trials, the mean cycles rises from 2 535 077 in group 1 ID to 4 189 661 in group 10 IDs which is an increase of about 65%. This difference is proportionally higher than the result from excluding the outliers, where a rise of about 28% is seen going from 1 926 346 to 2 463 945 in group 1 ID and 10 IDs respectively.

5.2.2 QEMU

In the full data results, QEMU underestimates the physical hardware by 25.6% at the smallest workload (1 ID). However, as the workload increases the relationship reverses and QEMU instead overestimates the cycles reaching a deviation of +22.8% for 5 IDs and +22.0% for 10 IDs. Instead looking at the data results without the outliers the cycles scales significantly faster, still underestimating the cycle counts at 1 ID but now by 18.7%. For 5 IDs it shows an overestimation of 51.4% and for 63.8% for 10 IDs. The crossing point and growing difference are visible in both Figures 5.1 and 5.2.

5.2.3 Wokwi

Wokwi's raw cycle counts are roughly 96% lower than the physical hardware across all workload sizes, a discrepancy traced to the cycle counter register being hard-coded to increment at 10 MHz rather than the configured 160 MHz CPU frequency. Applying a correction factor of $\times 16$ brings the adjusted means closer to the physical hardware. The data in table 5.1, the corrected deviations shows underestimations of 30.9% for 1 ID, 34.4% for 5 IDs and 49.4% for 10 IDs while the the data from table 5.2 shows underestimations of 16.9% for 1 ID, 21.7% for 5 IDs and 31.9% for 10 IDs. As seen in Figures 5.1 and 5.2, Wokwi consistently remains below the physical hardware across all workload sizes. Wokwi also shows the lowest standard deviation out of all the environments ranging from 516 464 to 855 392 cycles using all trials and 560 to 608 cycles without the outliers.

5.2.4 Comparison between emulators

Both emulators deviate more from the physical hardware as workload complexity increases, but in opposite directions, QEMU overestimates, while Wokwi underestimates. At 1 ID, the two emulators produce relatively similar deviations, but by 10 IDs their corrected means diverge significantly. The outlier counts follow similar patterns across all three environments, suggesting that the outlier spikes are at least partly a product of the workload itself, possibly due to interrupt-driven scheduling under FreeRTOS, rather than emulator-specific behavior. The outlier count follow a similar pattern across all the environments with the physical hardware and Wokwi showing the same amount of outliers.

5.3 Validity and Reliability

Regarding reliability, QEMU produced notably high standard deviations across all workload sizes, peaking at 154 953 cycles for the 5 ID workload when outliers are excluded. This implies that QEMU's results are not fully reproducible between trials, which is likely a consequence of its JIT-based translation. Wokwi on the other hand produced very low standard deviations, around 600 across all workloads when outliers are excluded.

Regarding validity, the outlier analysis reveals that all three environments produced a significant number of outliers, ranging from 77-317. The fact that outlier counts follow similar patterns across all environments suggest that these spikes are at least partly caused by the workload itself. This limits the internal validity of direct environment comparisons since an uncontrolled factor appears to influence all environments. Also as noted in Section 4.2.3, Wokwi was tested with a 4-byte UID while the other environments used a 7-byte UID, meaning that workloads are not fully identical. That is a threat to the internal validity of the comparison between Wokwi and the other environments



6 Discussion

In this section, the results presented in Chapter 5 will be discussed along with other factors of the thesis. The results will also be formulated to answer the research questions presented in Chapter 1. Some potential sources of error and limitation in the applied methodology are examined. This chapter puts the results into a broader context, considering implications for embedded systems development and the role of emulation in modern software engineering workflows.

6.1 Emulator Accuracy Relative to Physical Hardware

The first research question touched on how well the emulators could represent the ESP32-C3-Zero in the form of cycle count accuracy.

As presented in the results, QEMU deviated by 22.0-25.6% from the physical reference system. This does not align with what Böhm et al. [10] found while testing their JIT dynamic binary translation simulator, which was a typical variance of 1.4% and up to 5% in the worst cases. The deviation most likely attributed by the lack of well developed peripheral support in QEMU's Espressif fork, mostly due to the SPI interface used by the MFRC522 reader not existing at all and therefore the RFID read is triggered using a mock signal and sent via a separate function.

Wokwi, as shown in the results, deviated by 30.9-49.4% from the physical hardware, after applying the $\times 16$ correction for the 10 MHz counter. The shown result contradicts the hypothesis that a purpose-built embedded emulator with an existing MFRC522 model using emulated SPI would achieve higher cycle accuracy than QEMU. The MFRC522 model in Wokwi is probably modeled with prototyping and visualization in mind and not cycle accuracy.

It should also be noted that, as previously explained, Wokwi was tested with a 4-byte UID rather than the 7-byte UID used on the ESP32-C3-Zero and QEMU due to found limitations in the implementation Wokwi's MFRC522 model. A 4-byte UID read involves fewer SPI transactions than a 7-byte read resulting in Wokwi's workload not being entirely identical to the other environments.

6.2 Accuracy Difference Between Emulators

The second research question asked how different the emulators would behave compared to each other. Same as the previous question, the difference is represented by comparing the deviation in cycle accuracy.

At the smallest workload, using 1 ID, the two evaluated emulators produce comparable deviations from the physical reference system with QEMU at 25.6% and Wokwi at 30.9% for the full data results, a difference of 5.3 percentage points. As the workload increases the emulators diverges with QEMU ending up at a deviation of 22.0% and Wokwi at 49.4% with 10 IDs. This indicates that the two emulators model the workload in fundamentally different ways, probably mainly contributed to by Wokwi emulating a SPI interface while QEMU is not.

QEMU's standard deviation is significantly higher than Wokwi's across the board. In the results without the outliers, QEMU's standard deviation peaks at 154 953 cycles for 5 IDs while Wokwi's standard deviation remain below 40 cycles for each of the workloads. This consists with the JIT translation, explained in 3.2, that QEMU uses which means that the number of host instruction executed per target instructions can vary for each run. The variance shown here is noticeably higher than the 1.4% reported by Böhm et al. [10], this could be contributed by the added mock SPI function. The low variance in Wokwi's standard deviation is not by it self proof of good accuracy since it could be because the 10 MHz counter limits the resolution of the scaled results.

6.3 Contribution of Incomplete SPI Peripheral Modeling

The third and final research question asked how incomplete peripheral modeling, more specifically the SPI-based RFID communication, contributes to cycle count inaccuracy.

For QEMU, the SPI interface is entirely non-existent in the Espressif fork. The read operation is triggered by a mock function that injects a fixed byte sequence, bypassing any bus timing entirely. That means that the accuracy of the cycle count is completely up to how the developer constructs the mock function. This is probably one of the main contributors to the deviation from the ESP32-C3-Zero. This relates to findings by Chung et al. [7] that simplified peripheral models can cause deviations from physical hardware even when the CPU core model is otherwise modeled correctly.

Wokwi instead have an implemented model of the MFRC522 component, which means that SPI is modeled and the communications can happen via that interface. But as found, the MFRC522 RFID reader isn't fully modeled since it wouldn't accept UIDs larger than 4 bytes. This could also be a factor towards why Wokwi consistently underestimated the cycles of the ESP32-C3-Zero. Since Wokwi functionality under the hood is fairly undocumented, any reasons or claims to why a 7-byte UID wouldn't work could not be found.

6.4 Secondary Findings

During the testing phase of this study we found Wokwi to be an unreliable emulator when it comes to cycle count accuracy. This is mostly because of the generally undocumented nature of Wokwi's functionality makes it hard to find out what is actually happening during the simulation and how detailed the different parts of the emulation engine is modeled. The uncertainty of this is mostly contributed by Wokwi being a closed source application with a proprietary core.

At the first glance while testing the results in cycle count looked alarmingly low, with a factor of tens of thousands of cycles instead of millions found on both QEMU and the physical

hardware. With some further digging it seemed like the *mpccr* register in charge of holding the value of the cycle counts was only updated with an additional cycle every 16th cycle. This meant that even though the system was configured to run on 160 MHz the cycle counter was hard-coded to only run on 10 MHz. If the result then was multiplied by 16 the result was much more comparable to the results found running QEMU and the physical system. Even though this resulted in data closer to the compared results it doesn't give much confidence in Wokwi being an accurate emulator and the result being reliable.

6.5 Implications and Limitations

The results presented in this study are limited by the constraints described in Section 4.3. The used workload is a deterministic RFID read operation on one microcontroller architecture.

As previously noted in Section 4.3.1, cycle count does not paint the whole picture of the accuracy of a set emulator. Since Wokwi's cycle count register wasn't modeled correctly, another complementary metric would have been needed to further investigate its accuracy.

The not fully modeled SPI support in QEMU's Espressif fork is a notable constraint, this meant that the input signal from the MFRC522 reader needed to be imitated and written as a separate function inserting the data. This meant that the measured deviation was due to both CPU timing inaccuracies and gaps in the peripheral modeling.

6.6 Ethics and Sustainability

This section is meant to lift the ethical and environmental considerations found while conducting this study.

6.6.1 Ethical Considerations

The industrial context of this thesis, the RFID-based attendance tracking system, raises a few ethical concerns that should be mentioned. This is important even though the main focus of this thesis is the scientific context of embedded system emulation.

The most notable concern is student privacy. The system associates a student with a user ID (UID) from a physical card logging their presence at specific teaching moments, though in this early prototype the student is represented as only a number. If in the future the UID would be mapped to a LiU-ID or a name instead of a number this would fall under the scope of the General Data Protection Regulation (GDPR)¹, which is applied in Sweden. Real deployment would possibly require that the students are clearly informed of which data is collected and the purpose of the collection. Data of attendance linked to academic performance should be store in a secure way with restricted access by only authorized personnel.

Another concern raised by this thesis is the feasibility to clone the data on the card. While developing our attendance tracking system we found how easy it is to read and sort the data on these MIFARE-compatible cards, there exist multiple apps that can be downloaded on a smartphone with NFC that easily read and parse the information. Some of these apps even include a function to write data to a separate card, successfully cloning the original one. Our attendance tracking system could also be modified to do such things and with the saved data clone and use students LiU-card. This also could contribute to that students themselves clone cards to be able to present another students card while in a lecture, this could also be done by simply changing cards with each other.

¹<https://gdpr-info.eu/>

A more general ethical concern related to the findings of this thesis is the risk of relying on inaccurate emulators when developing safety-critical embedded systems that operates on a tight schedule. Embedded systems are often used in safety-critical applications such as medical equipment, industrial control systems, and infrastructure. The result shows that neither QEMU nor Wokwi provided reliable cycle-count accuracy, which makes their suitability in contexts where precise timing is critical questionable. If software developed and validated using an inaccurate emulator behaves differently on real hardware could the consequences range from system failure to life-threatening event.

6.6.2 Environmental Considerations

Emulation is a technique that let a developer work on a system without having access to the actual physical hardware, this leads to less hardware needing to be produced during the developing process. Less manufactured hardware results in less pollution and electronic waste which contributes to the project having a smaller environmental impact.

6.6.3 Economic Considerations

Emulation can lower development costs by removing the need for each developer or development team to have access to their own physical hardware. However the result of this thesis indicates that developers in need of cycle-accurate emulation should not rely on QEMU or Wokwi, as the inaccuracies found could mislead them and potentially result in them making a wrong decision about what microcontroller to choose. Developers that are in need of cycle-accurate emulation may therefore need to produce their own emulator rather than rely on open-source communities, which in itself is a cost that has to be weighed against simply purchasing the hardware.

6.6.4 Social Considerations

Emulators are a great way to let developers who cannot afford or access physical hardware to learn, experiment, prototype and contribute to embedded systems development. They lower the barrier to entry, which is particularly relevant in educational contexts or in regions where hardware is difficult to obtain. Emulation also allows people with disabilities that make hands-on work difficult, such as soldering, allowing them to still develop and learn within the field of embedded systems.

6.7 Method

The controlled experiment approach, characterized by Runeson and Höst [17] as a methodology that measures the effect of manipulating a fixed independent variable, the variable in this case being the environment, was an appropriate fit for this study. By changing the environment the dependent variable, cycle count, could be quantitatively and directly measured. The limitation of this approach is as mentioned in Section 4.1 that it trades realism for control, which in this case means that the conclusions are specified to the specific hardware, firmware and workload tested.

Cycle count is chosen as the main metric in this thesis for multiple reasons. First, it's directly observable with help of the earlier mentioned function. Second, it reflects timing behavior at the instruction level, which is what accuracy mean in the context of this thesis. Third, it is important for accuracy when developing an embedded system to have an emulator that is as cycle accurate as possible, but the cycle accuracy comes with the catch of higher computational cost as mentioned in Section 3.1.

The choice of doing 1000 trial for each environment on different levels of workloads was motivated by the need to obtain a stable mean despite the non-determinism observed on QEMU in the preliminary testing. A larger number of trials would improve this stability even further, but 1000 was determined to suffice. A thing worth mentioning is that in all three workloads tested on Wokwi the first logged measurement was found to be malformed, this left 999 usable trials to compare with the other environments.

The amount of logged IDs were considered sufficient to observe how emulators deviation scales with a increasing workload, but fall short of reflecting realistic deployment conditions. In practice the attendance tracking system described in Chapter 2 would be used in lecture settings where a typical class might range from around 40 students on the smaller side to upwards of 250 in the larger courses. Given that both emulators showed a clear trend of growing deviation with increased workload it is reasonable to expect that inaccuracies would be more pronounced at 40 or 250 IDs, though the exact scale remains unknown. Future work could therefore include workload sizes more closely reflecting realistic student body sizes as this would strengthen the external validity whether either emulator is suitable for development of a more realistic environment.

Wall-clock execution time was also briefly measured across the environments. However, the difference observed were small enough that they would not be perceptible to someone running the tested firmware. Since the workload is a relatively short and deterministic RFID read operation, the absolute execution times for the environments remained in a range that would not meaningfully affect the development workflow.



7 Conclusion

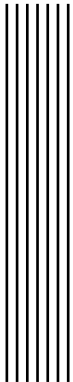
This thesis evaluated the accuracy of two embedded hardware emulators, QEMU [9] and Wokwi [11], by measuring the cycle counts of the runtime and compared it to an ESP32-based system.

Regarding the first research question how well embedded hardware emulators represent physical hardware in terms of cycle count accuracy the results show that neither emulator achieves consistent accuracy. QEMU deviated by -25.6% at the smallest workload but diverged to +22.0% at the largest, indicating that its inaccuracy scales with workload complexity. Wokwi, after correcting for the 10 MHz counter register anomaly, deviated between 30.9% and 49.4% depending on workload size. Both figures are broadly consistent with prior literature, where mean errors of 19–23% have been reported for RISC-V simulators and up to 5% for JIT-based binary translation approaches, though the QEMU overestimation at larger workloads exceeds those benchmarks.

Regarding the second research question how the two emulators differ from each other they diverge substantially both in direction and magnitude of error. At 1 ID the emulators produce comparable deviations, but by 10 IDs QEMU reports approximately 2.4 times the cycle count of the corrected Wokwi value. QEMU also exhibits significantly higher variability for each iteration, while Wokwi’s artificially low standard deviation is an artifact of the coarse counter resolution rather than genuine timing precision. A secondary finding is that Wokwi’s cycle counter register is hard-coded to operate at 10 MHz regardless of the configured CPU frequency, a limitation that is undocumented and that fundamentally undermines its reliability as a cycle-accurate emulator without applying a manual correction factor.

The primary limitation of this thesis is its narrow scope, one microcontroller running one workload measuring one metric. Future work should extend the methodology with support for additional hardware platforms, add more varied workloads and incorporate additional metrics. Wall-clock execution time may become a more informative metric for a larger and/or more complex workload where the emulator could take a significant amount of more time to run the same workload as its physical counterpart. As stated in Section 4.3.1, wall-clock execution time was measured in this study but produced negligible differences at the scale of

the firmware tested. More emulators should also be evaluated, especially focusing on cycle-accurate and cycle-approximate emulators.



Bibliography

- [1] Vahid Garousi, Michael Felderer, Çağrı Murat Karapıçak, and Uğur Yılmaz. “Testing embedded software: A survey of the literature”. In: *Information and Software Technology* 104 (2018), pp. 14–45. DOI: 10.1016/j.infsof.2018.06.016.
- [2] Pascal Pieper, Vladimir Herdt, and Rolf Drechsler. “Advanced Embedded System Modeling and Simulation in an Open Source RISC-V Virtual Prototype”. In: *Journal of Low Power Electronics and Applications* 12.4 (2022), p. 52. DOI: 10.3390/jlpea12040052.
- [3] Karan Pathak, Joshua Klein, Giovanni Ansaloni, Marina Zapater, and David Atienza. “Towards accurate RISC-V full system simulation via component-level calibration”. In: *ACM Transactions on Embedded Computing Systems* (2024). DOI: 10.1145/3737876.
- [4] Andrew Waterman. “Design of the RISC-V Instruction Set Architecture”. Tech. Rep. UCB/EECS-2016-1. PhD dissertation. University of California, Berkeley, 2016. URL: <https://people.eecs.berkeley.edu/~krste/papers/EECS-2016-1.pdf>.
- [5] Andrea Fernández Gallego, Miguel Jiménez Arribas, Iván Gamino del Río, Agustín Martínez Hellín, et al. “Count overflow and privilege mode filtering extension implementation on a RISC-V on-board processor”. In: *Microprocessors and Microsystems* 109 (2024). Open access, CC BY 4.0. Full author list: Fernández Gallego et al. (9 authors); verify on ScienceDirect before submission, p. 105084. DOI: 10.1016/j.micpro.2024.105084.
- [6] Miguel Jiménez Arribas, Agustín Martínez Hellín, Manuel Prieto, Iván Gamino del Río, et al. “Design and implementation of a synchronous Hardware Performance Monitor for a RISC-V space-oriented processor”. In: *Microprocessors and Microsystems* 112 (2025). Open access, CC BY 4.0. Full author list: Jiménez Arribas et al. (9 authors); verify on ScienceDirect before submission, p. 105132. DOI: 10.1016/j.micpro.2024.105132.
- [7] Sung Woo Chung, Gi Ho Park, Hyo-Joong Suh, Han Jong Kim, Jung Bin Im, Jung Wook Park, and Sung Bae Park. “Sim-ARM1136: A case study on the accuracy of the cycle-accurate simulator”. In: *Microprocessors and Microsystems* 30.3 (2006), pp. 137–144. ISSN: 0141-9331. DOI: <https://doi.org/10.1016/j.micpro.2005.07.002>. URL: <https://www.sciencedirect.com/science/article/pii/S0141933105000633>.

-
- [8] Shin-haeng Kang, Donghoon Yoo, and Soonhoi Ha. "TQSIM: A fast cycle-approximate processor simulator based on QEMU". In: *Journal of Systems Architecture* 66-67 (2016), pp. 33–47. ISSN: 1383-7621. DOI: <https://doi.org/10.1016/j.sysarc.2016.04.012>. URL: <https://www.sciencedirect.com/science/article/pii/S1383762116300297>.
 - [9] Fabrice Bellard. "QEMU, a fast and portable dynamic translator". In: *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*. 2005, pp. 41–46. URL: https://www.usenix.org/legacy/event/usenix05/tech/freenix/full_papers/bellard/bellard.pdf.
 - [10] Igor Böhm, Björn Franke, and Nigel Topham. "Cycle-accurate performance modelling in an ultra-fast just-in-time dynamic binary translation instruction set simulator". In: *2010 International Conference on Embedded Computer Systems: Architectures, Modeling and Simulation*. 2010, pp. 1–10. DOI: 10.1109/ICSAMOS.2010.5642102.
 - [11] Wokwi. *Wokwi Simulator Documentation*. <https://docs.wokwi.com>. Accessed: 2026. 2024.
 - [12] PlatformIO. *What is PlatformIO? PlatformIO Documentation*. <https://docs.platformio.org/en/latest/what-is-platformio.html>. Accessed: 2026. 2024.
 - [13] Frederic Leens. "An introduction to I2C and SPI protocols". In: *IEEE Instrumentation Measurement Magazine* 12.1 (2009), pp. 8–13. DOI: 10.1109/MIM.2009.4762946.
 - [14] Espressif Systems. *SPIFFS Filesystem ESP-IDF*. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/spiffs.html>. Accessed: 2026. 2024.
 - [15] V. Chawla and D. S. Ha. "An overview of passive RFID". In: *IEEE Communications Magazine* 45.9 (Sept. 2007), pp. 11–17. DOI: 10.1109/MCOM.2007.4342873.
 - [16] Anastasiia Butko, Rafael Garibotti, Luciano Ost, and Gilles Sassatelli. "Accuracy evaluation of GEM5 simulator system". In: *7th International Workshop on Reconfigurable and Communication-Centric Systems-on-Chip (ReCoSoC)*. 2012, pp. 1–7. DOI: 10.1109/ReCoSoC.2012.6322869.
 - [17] P. Runeson and M. Höst. "Guidelines for conducting and reporting case study research in software engineering". In: *Empirical Software Engineering* 14.2 (2009), pp. 131–164. DOI: 10.1007/s10664-008-9102-8.
 - [18] William Hellstrand and Hugo Holmberg. *Embedded Systems Emulation – Source Code*. <https://github.com/absquatulates/LIU-Attendance-Tracker>. Accessed: 2026. 2026.